

Real Estate Mining

Manual de Operacao do Sistema

Guia completo para iniciar, parar, monitorar e administrar a plataforma de mineracao de oportunidades imobiliarias em leiloes do Brasil e Estados Unidos.

ACESSO RAPIDO

URL: **http://localhost**

Email: **fabiano.freitas@gmail.com**

Senha: **260281xx**

Altere a senha apos o primeiro acesso

Sumario

1. Visao Geral da Arquitetura
2. Pre-requisitos e Diretorio de Trabalho
3. Iniciar o Sistema
4. Parar o Sistema
5. Reiniciar Servicos
6. Verificar Status
7. Acessar o Sistema (URLs e Credenciais)
8. Ver Logs e Monitoramento
9. Apos Mudancas noCodigo
10. Banco de Dados (backup e acesso)
11. Scraping Manual
12. Gerenciar Usuarios
13. Solucao de Problemas
14. Referencia Rapida (cheat sheet)

1. Visao Geral da Arquitetura

O sistema e composto por **8 servicos Docker** que trabalham juntos. Para o uso diario voce so precisa saber iniciar, parar e verificar o status.

Servico	Funcao	Porta
nginx	Porta de entrada — e o que o navegador acessa	:80
frontend	Interface visual (dashboard React)	:3000
api	Backend Django — processa dados e responde ao dashboard	:8000
postgres	Banco de dados — armazena todos os imoveis pesquisados	interno
redis	Fila de tarefas — coordena os workers	interno
celery_worker	Executa scraping e processamento de PDFs em background	interno
celery_beat	Agendador — dispara scraping automatico as 06:00	interno
wireguard	VPN — protege o IP durante scraping nos EUA	:51820

[i] **Uso diario:** acesse apenas `http://localhost`. Todos os servicos internos funcionam automaticamente.

2. Pre-requisitos e Diretorio de Trabalho

Confirme que estes programas estao instalados na maquina:

Software	Versao minima	Como verificar
Docker	24+	<code>docker --version</code>
Docker Compose	2.x (plugin)	<code>docker compose version</code>

Diretorio de trabalho

TODOS os comandos devem ser executados dentro do diretorio do projeto:

```
cd ~/Documents/real-estate-mining
```

[!] **Importante:** se voce executar os comandos fora desse diretorio, o Docker Compose nao encontrara os servicos.

3. Iniciar o Sistema

3.1 Iniciar tudo (uso normal)

Sobe todos os 8 servicos em background. Use este comando na maioria das vezes.

```
cd ~/Documents/real-estate-mining  
docker compose up -d
```

[v] Após alguns segundos acesse <http://localhost> no navegador.

3.2 Iniciar somente o essencial (sem scraping)

Economiza memoria quando nao precisar do scraping automatico:

```
docker compose up -d postgres redis api frontend nginx
```

[i] Neste modo o dashboard funciona normalmente. Scraping e processamento de PDFs ficam pausados.

3.3 Iniciar com logs visiveis (modo debug)

Para ver o que acontece em tempo real. Pressione **Ctrl+C** para parar:

```
docker compose up  
  
# Ctrl+C para parar e voltar ao terminal
```

4. Parar o Sistema

4.1 Parar tudo (dados preservados)

Para os containers mas **preserva o banco de dados e todos os arquivos**. Use sempre este comando.

```
docker compose down
```

[v] Os dados do banco e os PDFs baixados sao preservados. Pode reiniciar quando quiser.

4.2 Parar um servico especifico

Para apenas um servico sem afetar os demais:

```
docker compose stop api  
docker compose stop frontend  
docker compose stop celery_worker
```

[!] **NUNCA USE** `docker compose down -v` — esse comando apaga os volumes e voce **perdera TODOS os dados** do banco e PDFs.

5. Reiniciar Servicos

5.1 Reiniciar tudo

```
docker compose restart
```

5.2 Reiniciar um servico especifico

Util quando um servico travar ou apos alterar configuracoes:

```
docker compose restart api
docker compose restart frontend
docker compose restart celery_worker
docker compose restart celery_beat
docker compose restart nginx
```

5.3 Restart forçado (parar e subir)

Quando o restart simples nao resolve:

```
docker compose down
docker compose up -d
```

6. Verificar Status dos Servicos

6.1 Ver todos os servicos

```
docker compose ps
```

Na coluna **STATUS**, cada servico deve mostrar **Up**. Se aparecer **Exit** ou **Restarting**, ha um problema.

Status	Significado	O que fazer
Up	Funcionando normalmente	Nada
Up (healthy)	Funcionando e saudavel	Nada
Exit (0)	Parou sem erro	<code>docker compose start <servico></code>
Exit (1)	Parou com erro	<code>docker compose logs <servico></code>
Restarting	Reiniciando repetidamente	Ver logs imediatamente

7. Acessar o Sistema

Interface	URL	Quando usar
Dashboard	http://localhost	Uso diario — pesquisa de imoveis
Tela de login	http://localhost/login	Entrar na conta
Cadastro	http://localhost/register	Criar novo usuario
API REST	http://localhost:8000/api/	Desenvolvedores / debug
Admin Django	http://localhost:8000/admin/	Gestao avancada do banco

Credenciais de acesso

Campo	Valor
Email	fabiano.freitas@gmail.com
Senha	260281xx

[!] Altere a senha apos o primeiro acesso em <http://localhost/register> ou pelo Admin Django.

8. Ver Logs e Monitoramento

Logs sao o principal recurso para entender o que esta acontecendo e diagnosticar problemas.

8.1 Todos os servicos ao vivo

```
docker compose logs -f
# Ctrl+C para sair
```

8.2 Logs de um servico especifico

```
docker compose logs -f api
docker compose logs -f frontend
docker compose logs -f celery_worker
docker compose logs -f celery_beat
docker compose logs -f postgres
```

8.3 Ultimas N linhas (sem seguir ao vivo)

```
docker compose logs --tail=50 api
```

```
docker compose logs --tail=100 celery_worker
```

O que observar nos logs

Servico	Normal	Sinal de problema
api	Worker booted, GET/POST 200	Error, Exception, 500
celery_worker	Task received, Task succeeded	Task failed, Retry in
celery_beat	Sending due task (06:00)	ERROR, sem mensagens
postgres	Silencioso	FATAL, could not connect
nginx	GET / 200	502 Bad Gateway

9. Apos Mudancas noCodigo

Sempre que o desenvolvedor alterar arquivos do projeto, atualize os containers para as mudancas entrarem em vigor.

9.1 Mudanca em arquivos Python (backend)

```
docker compose restart api celery_worker celery_beat
```

9.2 Mudanca em arquivos React (frontend)

Em modo desenvolvimento o frontend atualiza automaticamente no navegador. Se nao atualizar:

```
docker compose restart frontend
```

9.3 Novas bibliotecas (requirements.txt ou package.json)

Quando novas bibliotecas sao adicionadas, e necessario rebuildar a imagem:

```
# Backend
docker compose build api
docker compose up -d

# Frontend
docker compose build frontend
docker compose up -d
```

9.4 Novas migrations do banco de dados

Quando o desenvolvedor alterar o modelo de dados, aplique as migrations:

```
docker compose run --rm api python manage.py migrate
```

10. Banco de Dados

10.1 Acessar o banco (modo interativo)

```
docker compose exec postgres psql -U postgres -d realestate_mining

# Para sair: \q + Enter
```

10.2 Fazer backup manual

```
docker compose exec postgres pg_dump -U postgres realestate_mining > backup_$(date +%Y%m%d).sql
```

[i] Salve o arquivo .sql em local seguro (Google Drive, HD externo). Faça isso regularmente.

10.3 Restaurar backup

```
docker compose exec -T postgres psql -U postgres realestate_mining < backup_20240101.sql
```

[!] Os dados ficam no volume Docker postgres_data. NUNCA use `docker compose down -v` — apaga o volume e todos os dados.

11. Scraping Manual

O scraping automatico roda todos os dias as **06:00**. Mas voce pode disparar manualmente quando precisar:

11.1 Rodar todos os scrapers agora

```
docker compose run --rm api python manage.py shell -c "  
from apps.scrapers.tasks import schedule_active_scrapers  
schedule_active_scrapers.delay()  
"
```

11.2 Processar PDF de um imovel manualmente

No dashboard, clique no card do imovel para abrir o painel de detalhe. O PDF sera exibido diretamente se ja foi processado. Caso contrario, o botao de processamento estara disponivel.

[i] Atencao: Para o scraping funcionar de verdade, os scrapers precisam ser configurados com as URLs reais dos condados. Esta e a proxima etapa do desenvolvimento.

12. Gerenciar Usuarios

12.1 Criar novo usuario (via dashboard)

Acesse <http://localhost/register> e preencha: nome, email e senha.

12.2 Resetar senha do admin padrao

```
docker compose run --rm api python manage.py seed_admin  
  
# Redefine: fabiano.freitas@gmail.com / 260281xx
```

12.3 Criar usuario via terminal

```
docker compose run --rm api python manage.py shell -c "  
from django.contrib.auth import get_user_model  
  
User = get_user_model()  
  
User.objects.create_user(username='email@ex.com', email='email@ex.com',  
password='senhaSegura123', first_name='Nome')  
"
```

12.4 Admin Django (gestao visual completa)

Em <http://localhost:8000/admin/> voce gerencia usuarios, imoveis e fontes de scraping visualmente com login de superusuario.

13. Solucao de Problemas

Problema: Dashboard nao abre (http://localhost sem resposta)

Solucao: Servicos parados. Verifique e suba novamente.

```
docker compose ps  
  
docker compose up -d
```

Problema: Login retorna "Credenciais invalidas"

Solucao: Admin com senha incorreta ou username errado. Execute o seed_admin.

```
docker compose run --rm api python manage.py seed_admin
```

Problema: API retorna erro 500

Solucao: Erro interno. Veja os logs para detalhes.

```
docker compose logs --tail=50 api
```

Problema: Banco de dados nao conecta

Solucao: Postgres parado. Inicie-o e reinicie a API.

```
docker compose up -d postgres  
  
docker compose restart api
```

Problema: Frontend mostra tela em branco

Solucao: Frontend ainda compilando. Aguarde 1-2 minutos e veja o log.

```
docker compose logs -f frontend  
  
# Aguarde: "Compiled successfully"
```

Problema: Scraping nao roda automaticamente

Solucao: Workers nao estao ativos. Suba-os.

```
docker compose up -d celery_worker celery_beat  
  
docker compose logs -f celery_beat
```

Problema: Mudanca no codigo nao apareceu

Solucao: Container ainda usa versao antiga. Rebuild ou restart.

```
docker compose build api  
  
docker compose up -d
```

14. Referencia Rapida — Cheat Sheet

Comandos mais usados no dia a dia:

Comando	O que faz
<code>docker compose up -d</code>	Iniciar tudo
<code>docker compose down</code>	Parar tudo (dados preservados)
<code>docker compose ps</code>	Ver status de todos os servicos
<code>docker compose restart api</code>	Reiniciar o backend
<code>docker compose restart frontend</code>	Reiniciar o frontend
<code>docker compose logs -f api</code>	Ver logs do backend ao vivo
<code>docker compose logs -f celery_worker</code>	Ver logs do scraping ao vivo
<code>docker compose build api</code>	Rebuildar imagem do backend
<code>docker compose build frontend</code>	Rebuildar imagem do frontend
<code>docker compose run --rm api python manage.py migrate</code>	Aplicar migrations
<code>docker compose run --rm api python manage.py seed_admin</code>	Resetar usuario admin

URLs de acesso rapido

<code>http://localhost</code>	Dashboard principal (uso diario)
<code>http://localhost/login</code>	Tela de login
<code>http://localhost/register</code>	Cadastro de novo usuario
<code>http://localhost:8000/api/</code>	API REST (desenvolvedores)
<code>http://localhost:8000/admin/</code>	Admin Django (gestao avancada)